



University of Hormozgan

Department of Computer Engineering

Design and Analysis of Algorithms

3Sum, Stable Matching,
Fibonacci Numbers, and Longest Consecutive Sequence

Dr. Golzari

Fall 1405 – 4051

University of Hormozgan

Contents

3Sum

Stable Matching

Fibonacci Numbers

Longest Consecutive Sequence

Conclusion

Roadmap

Four problems. Each looks simple, but the obvious approach is too slow.

- **3Sum** – when do three numbers cancel out to zero?
- **Stable Matching** – can everyone be paired so no two would rather switch?
- **Fibonacci Number** – how fast can we compute it?
- **Longest Consecutive Sequence** – how long is the longest run hiding in the array?

How we will work

Try your own approach first. We build the efficient algorithm together afterward.

Warm-up: The 2Sum Problem

Problem

Given an array A of integers and a target T , decide whether two elements of A sum to T .

- **Brute force:** check every pair $\Rightarrow O(n^2)$.
- **Hashing:** for each a_i , check whether $T - a_i$ was seen before $\Rightarrow O(n)$ time, $O(n)$ space.
- **Two pointers (if sorted):** start pointers at both ends, move inward $\Rightarrow O(n)$ time, $O(1)$ extra space.

Next: three numbers instead of two.

The 3Sum Problem

Problem

Given an array of integers `nums`, find all *unique* triplets (a, b, c) such that

$$a + b + c = 0.$$

Example:

`nums` = $[-4, -1, -1, 0, 1, 2]$

Answer: $\{(-1, -1, 2), (-1, 0, 1)\}$

Pause and Think

Think first

- Most direct way to find all triplets?
- How many candidate triplets, as a function of n ?

Brute Force: Check Every Triple

Three nested loops over all index triples $i < j < k$, testing $A[i] + A[j] + A[k] = 0$.

- Triples to check: $\binom{n}{3} = \Theta(n^3)$.
- Each check: $O(1)$.

Running time

$O(n^3)$ – for $n = 10^4$, about 10^{12} operations. Too slow.

Pause and Think

Where's the waste?

- Fixing a, b leaves: does $-(a + b)$ occur in the array? (2Sum again.)
- If the array were sorted, could you answer that without scanning everything?

Key Idea: Sort, Then Fix and Scan

- Sort `nums` in $O(n \log n)$.
- For each index i from left to right, fix $a = \text{nums}[i]$.
- The remaining task is: find b, c in `nums` $[i + 1..n - 1]$ with $b + c = -a$.
- This is exactly 2Sum on a *sorted* subarray $\Rightarrow$ solvable with two pointers in $O(n)$.

Total time

$O(n \log n)$ (sort) + $O(n) \cdot O(n)$ (two-pointer scans) = $O(n^2)$.

3Sum: Pseudocode

Input: Array $A[0..n-1]$

Output: All unique triplets summing to zero

```
1 Sort  $A$ 
2  $R \leftarrow \emptyset$ 
3 for  $i \leftarrow 0$  to  $n-3$  do
4   if  $i > 0$  and  $A[i] = A[i-1]$  then
5     continue
6    $lo \leftarrow i+1$ ,  $hi \leftarrow n-1$ 
7   while  $lo < hi$  do
8      $s \leftarrow A[i] + A[lo] + A[hi]$ 
9     if  $s = 0$  then
10      add  $(A[i], A[lo], A[hi])$  to  $R$ 
11       $lo \leftarrow lo+1$ ;  $hi \leftarrow hi-1$ 
12      skip duplicate values at  $lo$  and  $hi$ 
13   else if  $s < 0$  then
14      $lo \leftarrow lo+1$ 
15   else
16      $hi \leftarrow hi-1$ 
17 return  $R$ 
```

▷ skip duplicate anchors

3Sum: Python Implementation

```
1 def three_sum(nums):
2     nums.sort()
3     n, result = len(nums), []
4     for i in range(n - 2):
5         if i > 0 and nums[i] == nums[i - 1]:
6             continue # skip duplicate anchor
7         left, right = i + 1, n - 1
8         while left < right:
9             total = nums[i] + nums[left] + nums[right]
10            if total == 0:
11                result.append([nums[i], nums[left], nums[right]])
12                left, right = left + 1, right - 1
13                while left < right and nums[left] == nums[left - 1]:
14                    left += 1
15                while left < right and nums[right] == nums[right + 1]:
16                    right -= 1
17            elif total < 0:
18                left += 1
19            else:
20                right -= 1
21     return result
```

3Sum: Why the Two Pointers Never Miss a Solution

Fix i . Consider the sorted subarray $A[i + 1..n - 1]$ and pointers lo, hi .

- **Invariant:** any surviving valid pair $(A[lo'], A[hi'])$ satisfies $lo \leq lo' \leq hi' \leq hi$.
- If the sum is < 0 : $A[hi]$ can't be raised (sorted array), so only moving lo right can help – the old lo is safe to discard.
- Symmetric argument when the sum is > 0 .

Complexity

Sorting: $O(n \log n)$. Two-pointer scan per anchor: $O(n)$. **Total:** $O(n^2)$ time, $O(1)$ extra space (excluding output).

Worked Example: Tracing the Two Pointers

Sorted array: $[-4, -1, -1, 0, 1, 2]$, indices 0..5.

i (anchor a)	lo, hi start	what happens
0 (-4)	1, 5	every sum < 0 ; lo walks up to hi , no match
1 (-1)	2, 5	sum = 0 $\Rightarrow (-1, -1, 2)$; then 3, 4: sum = 0 $\Rightarrow (-1, 0, 1)$
2 (-1)	–	duplicate anchor, skipped
3 (0)	4, 5	sum = 3 > 0 ; pointers cross, no match

Result: $\{(-1, -1, 2), (-1, 0, 1)\}$ – matches the answer we guessed at the start.

Why Do We Care About Matching?

- Medical residents to hospitals (the NRMP "Match").
- Students to schools or dormitories.
- Kidney donors to compatible recipients.

Both sides have preferences. We want an assignment no pair would rather break to be with each other instead.

The Stable Matching Problem

Setup

n men and n women. Every person ranks all members of the opposite group in a strict order of preference.

- A **matching** M pairs every man with exactly one woman (a perfect matching).
- A pair (m, w) **not** in M is a **blocking pair** if m prefers w to his partner in M *and* w prefers m to her partner in M .
- M is **stable** if it has no blocking pair.

Question: does a stable matching always exist, and can we find one efficiently?

Try It Yourself

Men's preferences

$X : A \succ B \succ C$

$Y : B \succ A \succ C$

$Z : A \succ B \succ C$

Women's preferences

$A : Y \succ X \succ Z$

$B : X \succ Y \succ Z$

$C : X \succ Y \succ Z$

Pause and think

Pair them up by hand. Can you avoid any pair who'd rather be with each other instead of their assigned partner? Try more than once.

Why Not Brute Force?

Matching just three couples by hand already took some trial and error. Now imagine $n = 100$.

- $n!$ possible perfect matchings.
- Checking one matching for blocking pairs: $O(n^2)$.
- Total: $O(n! \cdot n^2)$ – infeasible.

Gale–Shapley (1962)

An $O(n^2)$ algorithm that always produces a stable matching, using a simple *proposal* process.

Gale–Shapley: Deferred Acceptance

- Every person starts **free**.
- While some man m is free and has a woman left to propose to:
 - ▶ m proposes to the next woman w on his list he has not yet proposed to.
 - ▶ If w is free, (m, w) become **engaged**.
 - ▶ If w is engaged to m' , she keeps the one she prefers and rejects the other.
 - ▶ The rejected man becomes free again and moves to the next woman on his list.
- Terminate when everyone is engaged.

Key fact: women only ever *trade up*, men only ever *propose down* their own list.

Gale–Shapley: Pseudocode

Input: Preference lists for n men and n women

Output: A stable matching

```
1 All men and women are free
2 while some man  $m$  is free and has not proposed to every woman do
3    $w \leftarrow$  next woman on  $m$ 's list not yet proposed to
4   if  $w$  is free then
5     engage ( $m, w$ )
6   else if  $w$  prefers  $m$  to her current partner  $m'$  then
7      $m'$  becomes free
8     engage ( $m, w$ )
9   else
10     $w$  rejects  $m$ 
11 return the set of engaged pairs
```

▷ m remains free

Worked Example: Resolving the Earlier Puzzle

Same instance as before. Let's run Gale–Shapley on it.

Men's preferences

$X : A \succ B \succ C$

$Y : B \succ A \succ C$

$Z : A \succ B \succ C$

Women's preferences

$A : Y \succ X \succ Z$

$B : X \succ Y \succ Z$

$C : X \succ Y \succ Z$

Trace:

1. $X \rightarrow A$: A free $\Rightarrow$ engage (X, A) .
2. $Y \rightarrow B$: B free $\Rightarrow$ engage (Y, B) .
3. $Z \rightarrow A$: A compares X vs. Z , prefers $X \Rightarrow Z$ rejected.
4. $Z \rightarrow B$: B compares Y vs. Z , prefers $Y \Rightarrow Z$ rejected.
5. $Z \rightarrow C$: C free $\Rightarrow$ engage (Z, C) .

Result: $\{(X, A), (Y, B), (Z, C)\}$ – stable. Same as your attempt?

Correctness (I): Termination

Claim

The algorithm terminates after at most n^2 proposals.

- Each man proposes to each woman **at most once** (his list only shrinks).
- There are n men and n women, so at most $n \cdot n = n^2$ proposals total.
- Each iteration of the loop performs one proposal $\Rightarrow$ the loop runs $O(n^2)$ times.

Correctness (II): Stability

Claim

The matching returned by Gale–Shapley has no blocking pair.

Proof by contradiction. Suppose (m, w) blocks M : m prefers w to $M(m)$, and w prefers m to $M(w)$.

- m prefers w to his own partner $\Rightarrow$ he proposed to w before reaching $M(m)$.
- So w rejected m at some point – for someone she preferred more.
- Women only trade up, so $M(w)$ is at least as good for her, hence w prefers $M(w)$ to m .
- Contradiction. ■

Correctness (III): Man-Optimality

Definition

w is a **valid partner** of m if some stable matching pairs them together.

Theorem

Gale–Shapley (men proposing) matches every man with his **best valid partner** – it is *man-optimal*.

- Consequently it is simultaneously **woman-pessimal**: every woman gets her worst valid partner among all stable matchings.
- Moral: *who proposes matters* – the proposing side has the advantage.

The Fibonacci Sequence

Definition

$$F(0) = 0, \quad F(1) = 1, \quad F(n) = F(n-1) + F(n-2) \quad (n \geq 2).$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Simple recurrence – but computing it *efficiently* teaches a lesson we'll reuse all term:
don't recompute the same subproblem.

Pause and Think

Before coding it up...

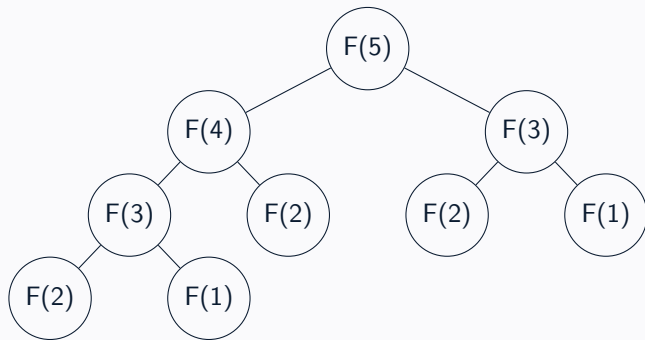
- Translate the definition directly into code – what does that look like?
- For $F(50)$, roughly how many calls would it make?

Naive Recursion: Exponential Blowup

```
1 def fib(n):  
2     if n <= 1:  
3         return n  
4     return fib(n - 1) + fib(n - 2)
```

- $T(n) = T(n - 1) + T(n - 2) + O(1)$ – same shape as Fibonacci itself.
- $T(n) = O(\varphi^n)$, $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$.
- **Root cause:** `fib(3)` gets recomputed many times over – overlapping subproblems.

The Recursion Tree



$F(3)$ and $F(2)$ each appear multiple times, recomputed from scratch.

Pause and Think

Look again at the tree

- Which values get computed more than once?
- What if we remembered a value the first time we compute it?

Fix 1: Memoization – Remember What We've Seen

Cache each result the first time it is computed.

```
1 def fib_memo(n, memo=None):
2     if memo is None:
3         memo = {}
4     if n in memo:
5         return memo[n]
6     if n <= 1:
7         return n
8     memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
9     return memo[n]
```

Complexity

Each of the $n + 1$ subproblems is solved exactly once in $O(1)$ work $\Rightarrow O(n)$ time, $O(n)$ space (memo table + recursion stack).

Fix 2: Building It Up, with $O(1)$ Space

Since $F(n)$ only depends on the two previous values, we do not need a full table.

```
1 def fib_dp(n):  
2     if n <= 1:  
3         return n  
4     prev, curr = 0, 1  
5     for _ in range(2, n + 1):  
6         prev, curr = curr, prev + curr  
7     return curr
```

Complexity

$O(n)$ time, $O(1)$ space – optimal in the number of arithmetic operations on the recurrence as stated.

Bonus: $O(\log n)$ via Matrix Exponentiation

$$\begin{pmatrix} F(n+1) \\ F(n) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n \begin{pmatrix} F(1) \\ F(0) \end{pmatrix}$$

- Repeated squaring computes M^n in $O(\log n)$ multiplications.
- Each is $O(1)$ arithmetic $\Rightarrow$ total $O(\log n)$.
- Same trick as fast exponentiation, $a^n \bmod p$.

Takeaway

$O(\varphi^n) \rightarrow O(n) \rightarrow O(\log n)$ – same problem, more structure exploited each time.

Problem Statement

Longest Consecutive Sequence

Given an unsorted array of integers, find the length of the longest run of **consecutive integers** (the numbers may appear in any order).

Example:

`nums = [100, 4, 200, 1, 3, 2]`

The sequence 1, 2, 3, 4 is the longest run $\Rightarrow$ answer = 4.

Pause and Think

Before reading on...

- Simplest way to find the longest run?
- Do you need a full order, or just "is $x + 1$ present"?

Naive Approach: Sort First

- Sort the array: $O(n \log n)$.
- Scan once, extending a run whenever consecutive elements differ by exactly 1 (skip exact duplicates): $O(n)$.

Running time

$O(n \log n)$ – correct, but the sort dominates the cost.

Can we drop the $\log n$?

- Sorting gives more than we need – a full order on every pair.
- We only ever ask "is $x - 1$ present?" or "is $x + 1$ present?"
- What structure answers "is y present?" in $O(1)$?

Key Idea: Only Start Counting at Sequence Starts

- Put all numbers into a hash set S .
- For a number x , it is the **start** of a run only if $x - 1 \notin S$.
- If x is a start, walk forward: $x, x + 1, x + 2, \dots$ while each is in S , counting the length.
- Numbers that are *not* a start are skipped entirely – they will be counted as part of the walk that starts before them.

Why is this $O(n)$?

Every number is walked over **at most once**, total, across the whole algorithm.

Longest Consecutive Sequence: Python

```
1 def longest_consecutive(nums):
2     num_set = set(nums)
3     longest = 0
4
5     for num in num_set:
6         if num - 1 not in num_set:      # num starts a sequence
7             length = 1
8             while num + length in num_set:
9                 length += 1
10            longest = max(longest, length)
11
12     return longest
```

Trace on [100, 4, 200, 1, 3, 2]: starts are 100, 200, 1 (since 99, 199, 0 $\notin S$). Walk from 1: 1, 2, 3, 4 $\Rightarrow$ length 4. Answer: 4.

Complexity Analysis

- Building the hash set: $O(n)$ time and space.
- Outer loop: $O(1)$ per element, unless it starts a walk.
- **Amortized:** every element belongs to exactly one run and is visited by exactly one walk – total work across all walks is $O(n)$.

Result

$O(n)$ expected time (hash set operations are $O(1)$ on average), $O(n)$ space.

Summary

Before vs. after:

Problem	Naive	Improved	Idea
3Sum	$O(n^3)$	$O(n^2)$	Sort + two pointers
Stable Matching	$O(n! \cdot n^2)$	$O(n^2)$	Deferred acceptance
Fibonacci Number	$O(\varphi^n)$	$O(n) / O(\log n)$	Memoize / matrix power
Longest Consec. Seq.	$O(n \log n)$	$O(n)$	Hash set

The recurring lesson

Speedups come from exploiting structure brute force throws away: sorted order, monotonic preferences, overlapping subproblems, $O(1)$ lookups.

Techniques Checklist

- **Two pointers:** sort first, then scan a monotone search space.
- **Proposing and accepting (Gale–Shapley):** irrevocable local choices that provably reach a stable outcome.
- **Remembering subproblem results:** cache, then trim history you no longer need.
- **Hashing:** trade space for $O(1)$ -average lookups instead of a $\log n$ sort.

Next session: Divide & Conquer and Graph Traversal.



Thank You

Design and Analysis of Algorithms

University of Hormozgan